

E-Touch OpenSDK documentation

Author	Date	Version	Modify/Add/Delete
Cris_Peng	2019/11/6	2.0.1.10	1. Add robot type query interface (3.7.10)
Cris_Peng	2019/10/28	2.0.1.10	1. Change of control protocol of Elf robot arm: changed from original HandMotionManager to WingMotionManager control, and the corresponding motion control class to RelativeAngleWingMotion, AbsoluteWingHandMotion, NoAngleWingMotion (corresponding protocol update such as 3.5 wing motion control) 2. The elf wheel no angle control protocol is modified: please see 3.6.1 for details. Mainly field modification 3. Compatible with D-type head motor control protocol. For D-type head motion control, please use DRelativeAngleHeadMotion, DAbsoluteAngleHeadMotion, and DNoAngleHeadMotion (the use of these three classes is consistent with the use in the D-type SDK document, only the class names have changed. For the elf version of head control, please see this article 3.4) HeadMotionManager correspondingly adds D-type robot control methods doDNoAngleMotion, doDAbsoluteAngleMotion, and doDRelativeAngleMotion
Cris_Peng	2019/6/3	2.0.1.8	1. Add desktop motion control protocol (3.15)
Cris_Peng	2019/3/7	2.0.1.7	1. The client heartbeat is maintained in the child thread to avoid being blocked by the main thread 2. Add desktop robot face tracking switch and status callback 3.14
Cris_Peng	2018/10/26	2.0.1.4	1. Add wheel motion callback (see 3.6.4 for details) 2. Add gyroscope data callback
Cris_Peng	2018/10/10	2.0.1.3	1. Add recognition language switching (3.2.16 is only applicable to foreign versions) 2. Add prohibition of semantic request pre-configuration (3.11.9 is only applicable to foreign versions) 3. Update the methods related to pulling video streams, remove the MediaManager management class and replace it with

			HDCameraManager
Cris_Peng	2018/9/12 2.0.1.2		1. Add anti-accidental killing configuration (3.11.8) 2. Method to hide the system floating button in the App (3.7.9)
Cris_Peng	2018/8/3	2.0.1.1	1. Added wake-up interface. Is it wake-up by voice or by other reasons (touch, call SpeechManager.doWakeUp method)? Please see the update changes 3.2.5 2. Added false recognition mode. Currently, the main control will actively intercept and filter the recognition of a word. After setting the false recognition mode, the main control will no longer intercept the recognition of a word (see 3.11.7 False Recognition Mode) 3. Added an interface to cancel the last recognition (currently for internal use) This update must be valid for
system versions after 2018/8/3	Cris_Peng	2018/6/21 2.0.1.0	The recognition interface returns to add recognition start, recognition end, recognition error Error return (only for domestic version) For details, please see 3.2.13, 3.2.14, 3.2.15

Table of contents

SDK Version Change Notes..... 7

1 Overview..... 10

2 Access Process..... 10

2.1 Development Environment..... 10

2.2 Configuration Instructions..... 11

2.2.1 aar version SDK configuration instructions..... 11

2.2.2 jar version SDK configuration instructions..... 11

2.3 Obfuscation Description..... 12

2.4 Notes..... 12

3 Detailed Introduction to Coded Access..... 12

3.1 Basic description of SDK.....	12
3.1.1 Description of Activity inheritance class.....	12
3.1.3 Special instructions for Type B Jar package.....	13
3.1.3 Description of the method return value OperationResult.....	14
3.2 Voice Control.....	14
3.2.1 Speech Synthesis.....	15
3.2.2 Speech Synthesis (Specifying Synthesis Parameters).....	15
3.2.3 Sleep.....	16
3.2.4 Wake-up.....	16
3.2.5 Wake-up Callback from Sleeping State.....	16
3.2.6 Recognition speech callback.....	17
3.2.7 Voice Volume Callback.....	18
3.2.8 Is the robot speaking?.....	18
3.2.9 Pause speech synthesis.....	19
3.2.10 Continue speech synthesis (overseas versions do not support this method).....	19
3.2.11 Stop speech synthesis (overseas version does not support this method).....	19
3.2.12 Speech Synthesis Status Callback.....	19
3.2.13 Identification start status callback.....	20
3.2.14 Identification end status callback.....	20
3.2.15 Identification Error Status Callback.....	21
3.2.16 Wake up and select the recognized language:	21
3.3 Hardware Control.....	22
3.3.1 Controlling White Light.....	22
3.3.2 Controlling LED Lights.....	23
3.3.3 Touch event callback.....	24

3.3.4 Sound source localization.....	25
3.3.5 PIR Detection.....	26
3.3.6 Infrared sensor callback (infrared ranging).....	26
3.3.7 Turn on/off black line filtering.....	27
3.3.8 Setting the Brightness of White Light.....	28
3.3.9 Gyroscope related function callbacks.....	28
3.3.10 Obstacle Detection.....	28
3.4 Head Movement Control.....	29
3.4.1 Relative angular motion.....	29
3.4.2 Absolute Angle Movement.....	30
3.4.3 Horizontal Center Lock.....	31
3.4.4 Absolute Angle Positioning Operation.....	31
3.4.5 Relative Angle Positioning Operation.....	32
3.5 Wing Motion Control.....	33
3.5.1 No Angle Motion.....	33
3.5.2 Relative angular motion.....	34
3.5.3 Absolute Angle Movement.....	35
3.6 Wheel Motion Control.....	36
3.6.1 No Angle Movement.....	36
3.6.2 Relative Angle Movement.....	37
3.6.3 Distance Movement.....	38
3.6.4 Wheel motion status callback.....	38
3.7 System Management.....	39
3.7.1 Obtaining the Device ID.....	39
3.7.2 Return to the screen saver animation interface.....	39

3.7.3 Expression Control.....	40
3.7.4 Obtaining Battery Power.....	41
3.7.5 Get Battery Status.....	41
3.7.6 Monitoring Security Home Alarms.....	41
3.7.7 Obtaining the Master Version Number.....	42
3.7.8 Show/Hide System Floating Button.....	42
3.7.9 Show/Hide the System Floating Button (to avoid flashing when jumping to Activity in APP)...	42
3.7.10 Query robot type.....	43
3.8 Multimedia Management.....	43
3.8.1 Get audio and video stream callback.....	43
3.8.2 Open Media Stream.....	44
3.8.3 Close Media Stream.....	45
3.8.4 Get Face Recognition Callback.....	45
3.8.5 Capturing images from video streams.....	46
3.9 Projector Control.....	47
3.9.1 Turning the Projector On and Off.....	47
3.9.2 Setting the Projector Mirror Image.....	47
3.9.3 Setting the Projector Keystone Horizontal Correction Value.....	48
3.9.4 Setting the Projector Keystone Vertical Correction Value.....	48
3.9.5 Setting the Projector Contrast.....	48
3.9.6 Setting the Projector Brightness.....	51
3.9.7 Setting the Projector Color Value.....	51
3.9.8 Setting the Saturation of the Projector Image.....	50
3.9.9 Setting the Projector Image Sharpness.....	50
3.9.10 Projector Expert Mode Optical Axis Adjustment.....	50

3.9.11 Projector Expert Mode Phase Adjustment.....	51
3.9.12 Setting the Projector Mode.....	51
3.9.13 Querying Projector Data and Status.....	51
3.9.14 Projector interface callback.....	52
3.10 Modular Motion Function Control.....	53
3.10.1 Turn on/off free walking.....	53
3.10.2 Turn on/off automatic charging.....	53
3.10.3 Get the free walking switch status.....	54
3.10.4 Obtaining the automatic charging switch status.....	54
3.11 Preprocessing Directives.....	55
3.11.1 Recording Switch.....	56
3.11.2 Identification Mode.....	56
3.11.3 Voice Mode.....	56
3.11.4 Touch Response Switch.....	57
3.11.5 PIR Response Switch.....	57
3.11.6 Voice Wake-up Response Switch.....	57
3.11.7 Misidentification Mode.....	57
3.11.8 Anti-accidental killing mode (the device will be killed when playing music in the background)...	57
3.11.9 Disable semantic request mode (only applicable to foreign versions).....	61
3.12 Command words and semantics.....	58
3.12.1 Customizing Global Command Words.....	61
3.12.2 Customized Semantics.....	60
3.13 Intelligent Peripheral Control.....	61
3.13.1 Obtaining the Whitelist.....	62
3.13.2 Determine whether Zigbee is ready.....	62

3.13.3 Get device list.....	62
3.13.4 Add to whitelist.....	63
3.13.5 Deleting the Whitelist.....	63
3.13.6 Enabling Whitelist.....	63
3.13.7 Setting the time when the device is allowed to join.....	63
3.13.8 Deleting a Device.....	64
3.13.9 Clearing the Blank List.....	64
3.13.10 Send Byte Command.....	64
3.13.11 Send String Command.....	64
3.13.12 Zigbee interface callback.....	64
3.14 Face Tracking and Active Greetings.....	65
3.14.1 Face Tracking Switch.....	65
3.14.2 Face tracking status callback.....	65
3.14.3 Active Greeting Switch.....	67
3.14.4 Active greeting status callback.....	67
3.15 Desktop Motion Control Protocol.....	68
3.15.1 Stop Movement.....	68
3.15.2 Desktop Motion Reset.....	68
3.15.3 Relative Angle Movement of Desktop.....	68
3.15.4 Absolute Angle Movement of Desktop.....	69

SDK version change notes

Version 2.0.1.10:

1. Added robot type query interface (3.7.10)

Version 2.0.1.9:

1. Change of control protocol of the Elf robot arm: from the original HandMotionManager to WingMotionManager control, the corresponding motion control class is changed to RelativeAngleWingMotion, AbsoluteWingHandMotion, NoAngleWingMotion (corresponding protocol updates such as 3.5 wing motion control) 2. Modification of the angle-free control protocol for the Elf wheels: For details, please see 3.6.1. The main modification is the constant modification of the motion mode 3. Compatible with the D-type head motor control protocol. For D-type head motion control, please use DRelativeAngleHeadMotion, DAbsoluteAngleHeadMotion, DNoAngleHeadMotion (the use of these three classes is consistent with the use in the D-type SDK document, only the class names have changed. For the head control of the Elf version, please see this article 3.4) HeadMotionManager correspondingly adds the D-type robot control methods doDNoAngleMotion, doDAbsoluteAngleMotion, doDRelativeAngleMotion

Version 2.0.1.8:

1. Add desktop robot control protocol (3.15)

Version 2.0.1.7:

1. The client heartbeat is maintained in the child thread to avoid being blocked by the main thread. 2. Add desktop robot face tracking switch and status callback (desktop robot)

Version 2.0.1.4:

1. Add wheel motion callback 2.
Add gyroscope data callback

Version 2.0.1.3:

1. Add recognition language switching (3.2.16 only applicable to foreign versions) 2.
Add prohibition of semantic request pre-configuration (3.11.9 only applicable to foreign versions)
3. Update the methods related to pulling video streams, remove the MediaManager management class, and replace it with HDCameraManager

Version 2.0.1.2:

1. Anti-killing configuration for background
music playback 2. Add another floating button hidden function, so that the activity jump in the same APP will not flash again

Version 2.0.1.1:

1. Add the wake-up interface to determine whether it is wake-up by voice or by other reasons (touch, call `SpeechManager.doWakeUp` method)
 2. Add the false recognition mode. Currently, the main control will actively intercept and filter the recognition of a word. After setting the false recognition mode, the main control will no longer intercept the recognition of a word 3.
- Add the interface to cancel the last recognition
- (currently for internal use)

Version 2.0.1.0:

1. Changed the SDK package name to `com.sanbot.opensdk` and the SDK name to `SanbotOpenSDK`.
2. Add recognition start, recognition end, and recognition error return interfaces (see 3.2.13, 3.2.14, 3.2.15 for details)

Version 2.0.0:

3. Modified the core communication mechanism of SDK and changed the writing language of SDK.
4. The communication mechanism of the projector has been modified, and some interfaces of the projector have been adjusted.
5. Added barrier detection interface.
6. Now the Activity also needs to call **the `register(Class subClass)` method** in the `onCreate` method to register the Activity's Class object with the SDK.

Version 1.1.8:

1. Added an interface for obtaining gyroscope sensor data.
 2. The relative angle movement and absolute angle movement of hand motion control increase the method of controlling the movement of two hands at the same time.
- Law.
3. Added the function of customizing global command words and customizing semantics.
 4. Corrected the touch event callback section and the document description of wheel non-angle movement.
 5. The speech recognition callback adds the open semantic function.
 6. The face recognition callback interface will now return all recognizable face data.
 7. Fixed the issue of projector interface failure.
 8. Fixed the bug that face recognition still triggers callback after the Activity is destroyed.
 9. Fixed the bug that ANR may occur when switching Activity pages when sending commands at a high frequency.
 10. Fixed the bug that SDK may cause the main controller to crash.

11. Added the interface for showing/hiding the system FloatBar function.

Version 1.1.7:

1. Added a modular motion manager to allow control of free walking and automatic charging functions.
2. Added the pre-processing command function to unify the previous recording, voice mode, and truncation function configurations.
to the preprocessor directive.
3. The power manager has been deleted, and the interface for obtaining the robot's power and charging status has been moved to the system manager.
4. Now the Service needs to call the register(Class subClass) method in the onCreate method to register the Service's Class object with the SDK.
5. Added pause, continue, stop speech synthesis, get the main control version number, get infrared ranging results, switch black
Interface for filtering lines, setting white light brightness, and monitoring security home alarms.
6. To be compatible with future semantic features, the interface for recognizing utterance callbacks has changed.

1 Overview

This document is intended for Android developers.

This document is used to guide developers to quickly access the Simbot open SDK. This SDK provides Android applications with functions such as controlling robot voice and actions.

Descriptions of robot names in the document: Model B = Elf; Model D = King Kong; Desktop robot = Mini Elf

2 Access Process

2.1 Development Environment

This SDK can be developed using development tools such as Android Studio or Eclipse. It is recommended that the software version of Android Studio be 1.5 or above.

JDK **version 7 or above** is required , and the minimum minSdkVersion of Android SDK is 11.

2.2 Configuration Instructions

In order to meet the development needs in different environments, we provide three different SDK versions:
Specifically: SanbotOpenSDK_XXX.aar, JarA, JarB.

The difference between JarA and JarB is that JarB does not provide BindBaseActivity, TopBindActivity and BindBaseService. These base classes need to use the BindBaseUtil class to implement communication with the robot.

2.2.1 aar version SDK configuration instructions

First, please install the provided SanbotOpenSdk_XXX.aar (XXX represents the version number) and gson-2.2.4.jar. Copy the file to the libs folder of the Module.

Add the following content to the build.gradle folder of the App Module:

```
repositories { flatDir
    { dirs 'libs'

    }
}
dependencies
    { compile(name:"SanbotOpenSdk_XXX", ext:'aar')
    }
```

2.2.2 jar version SDK configuration

instructions 1. Please put the JarA folder (or JarB folder, depending on whether you need to use A Jar package or B Jar package)

Copy all the files in the src/libs directory of the (Jar package) to the libs folder in your project.

2. Please merge the resource files in the src/res directory of the JarA (or JarB) folder with the resource files in your project.

3. Please add the following two permissions in your project's AndroidManifest.xml file:

```
<uses-permission android:name="android.permission.CAMERA" /> <uses-permission
android:name="android.permission.INTERNET"/> <uses-permission

android:name="android.permission.ACCESS_WIFI_STATE"/> <uses-permission

android:name="android.permission.ACCESS_NETWORK_STATE"/>
```

4. If you are using Android Studio for development, please remember to configure the path to read the so file.

Add the following content to the build.gradle folder of the Module:

```
sourceSets.main {

    jniLibs.srcDir 'libs'

}
```

2.3 Obfuscation Description

When integrating the application of the Sampo SDK, you need to pay attention to the methods related to the Sampo SDK cannot be obfuscated when packaging. The obfuscation method is as follows:

```
-keep class com.qihancloud.opensdk.**{*;}
-keep class com.sanbot.opensdk.**{*;}
-keep class com.sunbo.main.**{*;}
```

Ensure that the SDK class cannot be confused, otherwise there will be runtime exceptions such as failure to control the robot

2.4 Notes

1. At present, the development process needs to use the R&D system we provide for testing, and the official system needs to be in 2.21 and later

The version supports the existing functions.

2. The system does not include the Google service framework, which means that all services provided by Google cannot be used, such as Google Maps, Google

Login, etc. Developers should try to avoid using the corresponding SDK of Google Android Open Platform. 3. Please carefully monitor the system

startup broadcast. The sending time of the system startup broadcast has been changed to the first time you enter the system desktop after booting up. Please carefully

consider whether this will affect your program before using it. 4. The robot will automatically return to the desktop after the user stops

operating for a period of time. If you do not want your App to be returned to the desktop when it is open, please let your App call the Android system SDK to turn on

the screen always-on function. The reference code is as follows:

```
getWindow().setFlags(WindowManager.LayoutParams.FLAG_KEEP_SCREEN_ON,
WindowManager.LayoutParams.FLAG_KEEP_SCREEN_ON);
```

3. Detailed Introduction of Coded Access

3.1 Basic description of SDK

The SDK only supports controlling the robot in Activity and Service components.

3.1.1 Description of Activity inheritance class

All Activity classes in the project should inherit BindBaseActivity or TopBaseActivity, where TopBaseActivity is a subclass of BindBaseActivity. The similarities and differences between BindBaseActivity and TopBaseActivity are as follows:

Similarities: Both need to override an abstract method onMainServiceConnected() (which will only be called once when the Activity is created).

Differences: The page inherited from the TopBaseActivity class will have the system default status bar displayed at the top, and the layout is set. Please use the setContentView method when setting the Application style, and the Application style must adopt the NoActionBar style.

Note: If you want to control the robot immediately after the Activity is started, the corresponding control code must be placed in onMainServiceConnected(), otherwise it will be invalid in other life cycles! And you need to use register (XXActivity.Class) to register XXActivity in the OnCreate method, and the register method must be called before super.onCreate().

3.1.2 Description of Service inheritance class

If you need to control the robot in Service, you can inherit the BindBaseService class.

The BindBaseService class needs to overload an abstract method onMainServiceConnected() (which will only be called once when the Service is created), and register the XXService using **register(XXService.Class)** in the OnCreate method. **The register method must be called before super.onCreate();. If you need to control the robot as soon as the Service is started, the corresponding control code must be in onMainServiceConnected(), otherwise it will be invalid.**

Sample code:

```
public class SampleService extends BindBaseService {
    .....

    @Override
    protected void onMainServiceConnected() {
        control code
    }
    .....

    @Override
    public void onCreate() {
        register(SampleService.class);
        super.onCreate();
    }
    .....
}
```

3.1.3 Special instructions for Type B Jar package

Type B Jar package is an SDK that solves the problem of how to control the robot when developers cannot inherit from BindBaseActivity and BindBaseService. In Type B Jar package, the BindBaseUtil class is used instead. This class contains the following methods that need to be used:

1ÿBindBaseUtilÿActivity activityÿÿ BindBaseUtilÿService serviceÿ

The above two methods are the constructors of BindBaseUtil.

2ÿpublic void connectService()

Call this method to start establishing a connection with the robot master. Only after the connection is successfully established can you communicate with the robot. This method needs to be called in the `onResume()` life cycle method of the Activity or the `onCreate()` life cycle method of the Service.

3) `public void breakConnection()`

Call this method to disconnect from the robot master. This method needs to be called in the `onStop()` life cycle of the Activity. Method or Service's `onDestroy()` lifecycle method.

4) `public void setOnConnectedListener(OnConnectedListener onConnectedListener)`

A method to monitor whether a connection has been established with the master. When calling **the `connectService()` method to establish** a connection with the master, the connection process is an asynchronous operation, and this method will be called back after the connection is successful. If you need to perform certain operations immediately after a successful connection with the master, you can put the relevant code in this callback.

Please note: Please strictly follow the above requirements to put `connectService()` and `breakConnection()` into the life cycle of the corresponding component. Remember, do not forget to call the `breakConnection()` method, and do not call the `connectService()` and `breakConnection()` methods in life cycle methods that do not meet the requirements!!!

5) `public void register(Class subClass)`

Please call this method in the `Oncreate()` method of the Service and Activity components to register the class object of the current Service or Activity. This method must be called before the `super.onCreate()` method. If it is not registered or the registered object is wrong, the SDK's preprocessing instruction function may not work properly.

3.1.3 Description of the method return value `OperationResult` All

robot control methods will return a parameter of type **`OperationResult`**, which is used to feedback the result of the method execution: whether the execution is successful, the reason for the failure, and the return value after successful execution.

`OperationResult` has the following three methods:

`public int getErrorCode()` : instruction execution result, 1 indicates success, and other values less than 0 indicate execution failure.

`public String getDescription()`: Description of the command execution result. If the command fails to execute, The reason why the execution failed is described in detail.

`public String getResult()`: Additional field, generally used to store feedback data after instruction execution.

3.2 Voice Control

Access process:

1. Apply for a `SpeechManager` object. The application code is

```
SpeechManager speechManager = (SpeechManager)getUnitManager(FuncCo nstant.  
SPEECH_MANAGER);
```

2. Use the applied `speechManager` object to call the corresponding method for control.

3.2.1 Speech synthesis

method signature:

```
public OperationResult startSpeak(String text)
```

Method Description:

The speech synthesis method is used to control the robot to speak the specified text content. The language of the synthesized speech is related to the current system language setting.

Sample code:

```
speechManager.startSpeak("This is a test text");
```

3.2.2 Speech synthesis (specify synthesis parameters)

method signature:

```
public OperationResult startSpeak(String text,SpeakOption speakOption)
```

Method Description:

The overloaded method of the speech synthesis method can specify the language, speaking speed and intonation of the synthesized speech.

Parameter Description:

SpeakOption has the following parameters:

languageType indicates the language of the synthesized speech. The value range is LAG_CHINESE (Chinese), LAG_ENGLISH_US(English_United States).

speed indicates the speaking speed of the synthesized speech, and its value range is 0~100.

Intonation represents the tone of synthesized speech, and its value range is 0~100.

Sample code:

```
SpeakOption speakOption = new SpeakOption();  
speakOption.setSpeed(70);  
OperationResult operationResult = speechManager.startSpeak("This is a test sentence",speakOption);
```

3.2.3 Hibernate

method signature:

```
public OperationResult doSleep()
```

Method Description:

Put the machine into sleep mode and stop talking.

Sample code:

```
speechManager.doSleep();
```

3.2.4 Wake-up

method signature:

```
public OperationResult doWakeUp()
```

Method Description:

Actively wake up the device. This method can only be called in an Activity. It will not take effect if called in a Service.

Sample code:

```
speechManager.doWakeUp();
```

3.2.5 Waking up the dormant state callback

method signature:

```
public void setOnSpeechListener(SpeechListener speechListener)
```

Method Description:

setOnSpeechListener is a general callback interface for voice control. The parameters passed determine the callback interface to which function the callback belongs. This callback method will be triggered when the robot enters the sleep state or wakes up state.

Sample code:

```
speechManager.setOnSpeechListener(new WakenListener() {  
    @Override  
    public void onWakeUp() { //Wake-up  
        event callback  
    }  
})
```

```
@Override  
public void onSleep() { //Sleep event  
    callback  
}  
  
@Override  
public void onWakeUpStatus(boolean b) { //Wake-up event  
    callback b==true means the event is woken up by the wake-up word  
}  
});
```

3.2.6 Recognition speech callback

method signature:

```
public void setOnSpeechListener(SpeechListener speechListener)
```

Method Description:

Please note: To be compatible with subsequent semantic features, this interface has changed since 1.1.7.

This interface provides all the text recognized by the robot except the wake-up word to the third-party application.

In the third-party application, you can decide whether to enable the truncation function (the truncation function is to prevent the robot from identifying the text

Please note that the robot will not recognize the user's words when it is in sleep mode.

By default, the truncation function is disabled. If you need to enable the truncation function for the current activity,

Please add the following preprocessing instructions to the AndroidManifest.xml file (for instructions on preprocessing instructions, refer to Section 3.11):

```
<meta-data android:name="RECOGNIZE_MODE" android:value="1"/>
```

Note: The execution time of the code in the onRecognizeResult callback method should not exceed 500ms, otherwise it will affect

It will affect the normal operation of the entire system. It is not recommended to perform truncation operations in Service.

Parameter Description:

The robot recognition speech callback implements the RecognizeListener interface. In this interface, the following methods need to be implemented:

```
boolean onRecognizeResult(Grammar grammar);
```

Users can use the grammar.getText() method to obtain the text recognized by the robot.

The onRecognizeResult return value is a Boolean type. The return value is determined by the developer based on actual business needs.

When the return value is true, it means the robot will no longer respond to the text, and false otherwise.

The Grammar object contains the content of the user's speech recognized by the robot and the interpretation of the user's speech.

The result of semantic parsing. Currently, semantics only supports systems that use the iFlytek semantic engine. For a description of the Grammar class, refer to the Sanbao Open Semantic Description Document.

Sample code:

```
speechManager.setOnSpeechListener(new RecognizeListener() {  
    @Override  
    public boolean onRecognizeResult(Grammar grammar) {  
        System.out.print("The text recognized by the robot is " + grammar.getText());  
        return true;  
    }  
});
```

3.2.7 Voice volume callback

method signature:

```
public void setOnSpeechListener(SpeechListener speechListener)
```

Method Description:

When the robot recognizes the sound of the surrounding environment, this method will be called back and the volume of the recognized sound will be returned. This method can be used to determine the user's speaking status and obtain the volume of the user's speaking voice. **This method will only be called back when the robot is in the awake state.**

Parameter Description:

The robot recognition speech callback implements the RecognizeListener interface. In this interface, the following methods need to be implemented:

```
void onRecognizeVolume(int volume);
```

The volume value returned is the volume of the sound recognized by the robot, ranging from 0 to 30.

3.2.8 Is the robot speaking? Method signature:

```
public OperationResult isSpeaking()
```

Method Description:

This method is used to determine whether the current robot is speaking.

Return value description:

The getResult() method of OperationResult will return the query result. If the returned value is "1", it means the robot is speaking, and "0" means otherwise.

Sample code:

```
OperationResult operationReusult = speechManager.isSpeaking ();  
  
if(operationResult.getResult().equals("1")){ //TODO robot is  
speaking }
```

[3.2.9 Pause speech synthesis](#)

method signature:

```
public OperationResult pauseSpeak()
```

Method Description:

This method is used to pause the speech synthesis of the current robot.

[3.2.10 Continue speech synthesis \(overseas versions do not support this method\)](#)

method signature:

```
public OperationResult resumeSpeak()
```

Method Description:

This method is used to resume the currently paused speech synthesis of the robot.

[3.2.11 Stop speech synthesis \(overseas version does not support this method\)](#)

method signature:

```
public OperationResult stopSpeak()
```

Method Description:

This method is used to terminate the speech synthesis of the current robot.

[3.2.12 Speech synthesis status callback](#)

method signature:

```
public void setOnSpeechListener(SpeechListener speechListener)
```

Method Description:

This callback method will be triggered when the robot is performing speech synthesis.

Parameter Description:

The speech synthesis status callback implements the SpeakListener interface. In this interface, the following methods need to be implemented

Law:

```
void onSpeakStatus(SpeakStatus speakStatus);
```

The onSpeakStatus method will be called back during the speech synthesis process. The progress parameter of the SpeakStatus class indicates the current synthesis progress of the sentence, and the value range is 0<=percent<=100.

Sample code:

```
speechManager.setOnSpeechListener(new SpeakListener() {  
  
    @Override  
    public void onSpeakStatus(SpeakStatus speakStatus)  
        { System.out.print("Speech is being synthesized. The  
        progress is : " + speakStatus.getProgress());  
        }  
});
```

[3.2.13 Identify the start state callback method](#)

signature:

```
public void setOnSpeechListener(SpeechListener speechListener)
```

Method Description:

This method will be called back when the robot starts recognition

Parameter Description:

The robot recognition speech callback implements the RecognizeListener interface. In this interface, the following methods need to be implemented:

```
@Override  
public void onStartRecognize() { Log.  
    i ("Cris", "Status callback when recognition starts");  
}
```

[3.2.14 Identification end status callback](#)

method signature:

```
public void setOnSpeechListener(SpeechListener speechListener)
```

Method Description:

The robot will call back this method when it finishes recognition.

Parameter Description:

The robot recognition speech callback implements the RecognizeListener interface. In this interface, the following methods need to be implemented:

```
@Override
public void onStopRecognize() { Log.
    i ("Cris", "Status callback when recognition ends");
}
```

3.2.15 Identify the error status callback

Method signature:

```
public void setOnSpeechListener(SpeechListener speechListener)
```

Method Description:

This method will be called back when the robot recognizes an error

Parameter Description:

The robot recognition speech callback implements the RecognizeListener interface. In this interface, the following methods need to be implemented:

```
@Override
public void onError(int engine, int errorCode) {
    Log. i ("Cris", "onError: engine =" + engine + " errorCode =" + errorCode);
}
```

Engine The current recognition engine. 0 represents iFlytek, 1 represents DuerOS.

ErrorCode The error code identified. For example, when engine==0, errorCode==20005 means that no one is talking at the moment. For other error codes, please check the error code list of iFlytek.

3.2.16 Wake up and select the recognized language:

Method signature:

```
public OperationResult doWakeUp(WakeUpOption option)
```

Method Description:

Actively wake the machine up. This method can only be called in an Activity. It will not work if called in a Service. Foreign customers need to select different languages for recognition when waking up. In addition to switching the system default language, a new WakeUpOption is now added when waking up. WakeUpOption can configure the robot recognition language.

Sample code:

WakeUpOption currently supports the following languages:

LAG_ARABIC_INTERNATIONAL //Arabic

LAG_CHINESE_HK //China Hong Kong

LAG_DANISH //Danish

LAG_ENGLISH_US //English

LAG_FRENCH_FRANCE // French

LAG_GERMAN //German

LAG_ITALIAN //Italian

LAG_JAPANESE //Japanese

LAG_KOREAN //Korean

LAG_POLISH //Polish

LAG_PORTUGUESE_PORTUGAL //Portuguese

LAG_SPANISH_SPAIN //Spanish

LAG_TURKISH // Turkish

```
WakeUpOption wakeUpOption = new WakeUpOption();
wakeUpOption.setLanguageType(WakeUpOption.speechManager.doWakeUp(wakeUpOption);
```

3.3 Hardware Control

Access process:

1. Apply for a `HardwareManager` object. The application code is

```
HardwareManager hardwareManager = (HardwareManager)getUnitManager
(FuncConstant.HARDWARE_MANAGER);
```

2. Use the applied `hardwareManager` object to call the corresponding method for control.

3.3.1 Controlling white light

Method signature:

```
public OperationResult switchWhiteLight(boolean isOpen)
```

Method Description:

Used to turn the white light on or off.

Sample code:

```
hardWareManager.switchWhiteLight(true);
```

3.3.2 Control LED light

method signature:

```
public OperationResult setLED(LED led)
```

Method Description:

Control the LED lights of the robot.

Parameter Description:

The LED class is constructed as follows:

```
LED(byte part, byte mode, byte delayTime, byte randomCount)
```

The part parameter indicates the location of the LED light on the robot.

The mode parameter indicates the control mode of the LED light.

delayTime is only used when the LED is in flashing mode, indicating the flashing interval of the LED (please

Note that the value range of delayTime is between 1 and 255, and the unit is 100ms).

randomCount is only used when the LED light is in random color flashing mode, indicating the random color of the flashing

Quantity (randomCount value range is 1~7).

The following are possible values for the part parameter:

LED.PART_ALL All LED lights of the robot

LED.PART_WHEEL LED lights on the robot chassis

LED.PART_LEFT_HAND LED light for the robot's left wing

LED.PART_RIGHT_HAND LED light on the right wing of the robot

LED.PART_LEFT_HEAD LED light on the left side of the robot head

LED.PART_RIGHT_HEAD LED light on the right side of the robot head

The following are the possible values passed by the mode parameter:

LED.MODE_CLOSE turns off the LED light

LED.MODE_WHITE White LED light

LED.MODE_RED Red LED light

LED.MODE_GREEN Green LED light

LED.MODE_PINK Pink LED light

LED.MODE_PURPLE Purple LED light

LED.MODE_BLUE blue LED light

LED.MODE_YELLOW Yellow LED light

LED.MODE_FLICKER_WHITE flashing white LED light (**delayTime** can control the LED
Flashing interval time)

LED.MODE_FLICKER_RED Flashing red LED

LED.MODE_FLICKER_GREEN Flashing green LED light

LED.MODE_FLICKER_PINK Flashing pink LED light

LED.MODE_FLICKER_PURPLE flashes purple LED light

LED.MODE_FLICKER_BLUE flashing blue LED light

LED.MODE_FLICKER_YELLOW Flashing yellow LED light

LED.MODE_FLICKER_RANDOM Random color flashing (**randomCount** can control the number of colors that the
LED flashes randomly)

Sample code:

```
hardWareManager.setLED(new LED(LED.PART_ALL,LED.  
MODE_FLICKER_RANDOM,10,3));
```

3.3.3 Touch event callback

method signature:

```
public void setOnHareWareListener(HardWareListener hareWareListener)
```

Method Description:

When the robot's touch sensor is triggered, a callback will be generated for the touch event.

Parameter Description:

The touch event implements the TouchSensorListener interface, in which the following methods need to be implemented:

```
void onTouch(int part);
```

The value range of part is 1~13.

1	Right chin sensor	2	Left chin sensor
3	Right chest sensor	4	Left chest sensor
5	Left side sensor at the back of the head	6	Sensor on the right side of the back of the head
7	Back left sensor	8	Back right sensor
9	Left hand sensor	10	Right hand sensor
11	Sensor in the middle of the head	12	Overhead right front sensor
13	Overhead left front sensor		

Sample code:

```
hardWareManager.setOnHareWareListener(new TouchSensorListener() {  
    @Override  
    public void onTouch(int part) {  
        if (part == 11 || part == 12 || part == 13) {  
            touchTv.setText("You touched the head");  
        }  
    }  
});
```

3.3.4 Sound source localization

Method signature:

```
public void setOnHareWareListener(HardWareListener hareWareListener)
```

Method Description:

When the robot is awakened by voice, a callback will be generated for the sound source localization event.

Parameter Description:

The sound source localization event implements the VoiceLocateListener interface. In this interface, the following methods need to be implemented:

```
void voiceLocateResult(int angle);
```

Angle represents the offset angle of the sound source relative to the front of the head and is calculated in a clockwise direction.

Sample code:

```
hardWareManager.setOnHareWareListener(new VoiceLocateListener() {  
    @Override  
    public void voiceLocateResult(int angle) {  
        //TODO Get the sound source positioning angle and process it  
    }  
});
```

3.3.5 PIR detection

method signature:

```
public void setOnHareWareListener(HardWareListener hareWareListener)
```

Method Description:

There is a PIR module in front and behind the robot. When the robot PIR module detects someone coming from the robot

When a person passes by or leaves the front or back, a PIR detection event will be called back.

Parameter Description:

The PIR detection event implements the PIRListener interface, in which the following methods need to be implemented:

```
void onPIRCheckResult(boolean isChecked,int part);
```

When isChecked is true, it means that the PIR detects that someone is passing by. When it is false, it means that someone is detected leaving the robot from the front or back. The part parameter indicates the location of the infrared sensor, and the value range is 1~2. 1 indicates the PIR in front of the robot, and 2 indicates the PIR behind the robot.

Sample code:

```
hardWareManager.setOnHareWareListener(new PIRListener() {  
    @Override  
    public void onPIRCheckResult(boolean isCheck,int part) {  
        System.out.print((part == 1 ? "front" : "back") + "PIR is triggered");  
    }  
});
```

3.3.6 Infrared sensor callback (infrared ranging) method

signature:

```
public void setOnHareWareListener(HardWareListener hareWareListener)
```

Method Description:

The infrared sensor callback function currently returns the distances to obstacles measured by the robot's 18 infrared emitters.

The schematic diagram of the robot's infrared transmitter position is as follows:



Parameter Description:

The infrared sensor related functions are implemented by the InfrareListener interface. In this interface, the following methods need to be implemented:

```
void infrareDistance(int part,int distance);
```

infrareDistance is a method for infrared ranging results. The part parameter indicates the infrared sensor part. The specific value corresponds to the part marked in the figure above. The value range is 1 to 17, indicating infrared ranging modules 1 to 17 respectively. distance indicates the distance from the measuring module to the obstacle, in centimeters.

Sample code:

```
hardWareManager.setOnHareWareListener(new InfrareListener() {  
  
    @Override  
    public void infrareDistance(int part, int distance) { if (distance != 0)  
        { System.out.print("part" + part  
            + "distance is" + distance); }  
  
    } });
```

[3.3.7 Turn on/off black line filtering](#)

method signature:

```
public OperationResult switchBlackLineFilter(boolean isOpen)
```

Method Description:

If there are wide black lines or gaps on the ground, it will affect the normal operation of the robot's infrared ranging function, thereby stopping the movement of the robot's wheels. If you need to prevent the robot from being disturbed in this environment, you can continue to

If the robot continues to move, you need to turn on the black line filter function. Please note: turning on the black line filter function will cause the robot's anti-fall function to fail. Please avoid turning on the black line filter function in an environment where there may be a risk of the robot falling.

3.3.8 Set the white light brightness

method signature:

```
public OperationResult setWhiteLightLevel(int level)
```

Method Description:

Set the brightness of the robot's white light. The value range of level is 1~3, representing the brightness level of the white light as energy-saving, soft, and bright.

3.3.9 Gyroscope related function callback

method signature:

```
public void setOnHareWareListener(HardWareListener hareWareListener)
```

Method Description:

This callback interface is used to obtain the deflection angle data measured by the gyroscope device on the robot.

Parameter Description:

The gyroscope-related functions are implemented by the GyroscopeListener interface. In this interface, the following methods need to be implemented:

```
void gyroscopeData(float driftAngle, float elevationAngle, float rollAngle);
```

The above method is a callback of the gyroscope measurement result. driftAngle represents the yaw angle data, elevationAngle represents indicates pitch angle data, and rollAngle indicates roll angle data.

Sample code:

```
hardWareManager.setOnHareWareListener(new
GyroscopeListener()
{ @Override
    public void gyroscopeData(float driftAngle, float
elevationAngle, float rollAngle) {
        //TODO return gyroscope measurement data}
});
```

3.3.10 Obstacle detection

method signature:

```
public void setOnHareWareListener(HardWareListener hareWareListener)
```

Method Description:

The robot's wheel chassis, wings, and front and back of the body are equipped with distance detectors. When these sensors detect obstacles nearby, they will report the obstacle status.

Parameter Description:

The obstacle detection event implements the `ObstacleListener` interface. In this interface, the following methods need to be implemented:

`void onObstacleStatus(boolean status);`

Callback parameter status: true, in obstacle avoidance false, environment safe

Sample code:

```
hardwareManager.setOnHareWareListener(new
ObstacleListener()
{ @Override
public void onObstacleStatus(boolean b) { if (b)

    { Log.w("info", "onObstacleStatus: Avoiding obstacles!"); } else
    { Log.i("info",
        "onObstacleStatus: No obstacles around.");
    }
}
});
```

3.4 Head Movement Control

Access process:

1. Apply for a `HeadMotionManager` object. The application code is

```
HeadMotionManager headMotionManager= (HeadMotionManager)getUnitManager(FuncConstant.HEADMOTION_MANAGER); 2. Use the applied
headMotionManager object to call the corresponding method for control.
```

3.4.1 Relative angle motion

method signature:

`public OperationResult doRelativeAngleMotion(RelativeAngleHeadMotion relativeAngleHeadMotion)`

Method Description:

Control the robot head to rotate a certain angle in the specified direction relative to the current position.

Parameter Description:

The **RelativeAngleHeadMotion** class has two parameters, action and angle.

Human head movement mode, angle represents the relative angle of the robot's head movement.

The following are the possible values that can be passed to the action parameter:

RelativeAngleHeadMotion.ACTION_STOP Stop motion

RelativeAngleHeadMotion.ACTION_UP Upward movement

RelativeAngleHeadMotion.ACTION_DOWN downward movement

RelativeAngleHeadMotion.ACTION_LEFT Move left

RelativeAngleHeadMotion.ACTION_RIGHT Move to the right

RelativeAngleHeadMotion.ACTION_LEFTUP Move to the upper left

RelativeAngleHeadMotion.ACTION_RIGHTUP Move to the upper right

RelativeAngleHeadMotion.ACTION_LEFTDOWN moves to the lower left

RelativeAngleHeadMotion.ACTION_RIGHTDOWN moves to the lower right

Sample code:

```
RelativeAngleHeadMotion relativeAngleHeadMotion = new
RelativeAngleHeadMotion(
    RelativeAngleHeadMotion.ACTION_RIGHT,30
);
headMotionManager.doRelativeAngleMotion(relativeAngleHeadMotion);
```

[3.4.2 Absolute angle movement](#)

method signature:

public OperationResult doAbsoluteAngleMotion(AbsoluteAngleHeadMotion absoluteAngleHeadMotion)

Method Description:

Control the robot head to rotate to the position of the specified angle. The absolute angle value range of the horizontal direction is 0~180

The absolute angle in the vertical direction ranges from 7 to 30 degrees, and the angle value increases from the bottom to the top.

Parameter Description:

The **AbsoluteAngleHeadMotion** class has two parameters, action and angle.

Human head movement mode, angle represents the absolute angle of the robot's head movement.

The following are the possible values that can be passed to the action parameter:

AbsoluteAngleHeadMotion.ACTION_VERTICAL Vertical movement

AbsoluteAngleHeadMotion.ACTION_HORIZONTAL horizontal movement

Sample code:

```
AbsoluteAngleHeadMotion absoluteAngleHeadMotion = new
AbsoluteAngleHeadMotion(
    AbsoluteAngleHeadMotion.ACTION_HORIZONTAL,130
);
headMotionManager.doAbsoluteAngleMotion(absoluteAngleHeadMotion);
```

3.4.3 Horizontal center lock

method signature:

public OperationResult dohorizontalCenterLockMotion()

Method Description:

Control the robot head to rotate to a horizontal position of 90 degrees, and lock the robot's horizontal motor. (After locking, the robot head cannot be rotated horizontally manually).

Sample code:

```
headMotionManager.dohorizontalCenterLockMotion();
```

3.4.4 Absolute angle positioning operation

method signature:

public OperationResult

doAbsoluteLocateMotion(LocateAbsoluteAngleHeadMotion locateAbsoluteAngleHeadMotion)

Method Description:

Control the robot's head to rotate to the specified angle, and decide whether to turn on the head motor after completing the action.

The absolute angle value range of the horizontal direction is 0~180 degrees, and the angle value increases from left to right. The absolute angle value

range of the vertical direction is 7~30, and the angle value increases from the bottom to the top. The difference between this method and the head

absolute angle movement method is that this method can decide whether to lock the motor. This method does not need to specify the rotation mode, but

directly specifies the absolute angle values of the horizontal and vertical directions.

Parameter Description:

The **LocateAbsoluteAngleHeadMotion** class has three parameters: action, horizontalAngle

action indicates the robot head lock mode, horizontalAngle indicates the absolute horizontal angle of the robot head movement, and verticalAngle indicates the absolute vertical angle of the robot head movement.

The following are the possible values that can be passed to the action parameter:

LocateAbsoluteAngleHeadMotion.ACTION_NO_LOCK The motor does not lock

LocateAbsoluteAngleHeadMotion.ACTION_HORIZONTAL_LOCK The motor is locked in the horizontal direction

LocateAbsoluteAngleHeadMotion.ACTION_VERTICAL_LOCK The motor is locked in the vertical direction

LocateAbsoluteAngleHeadMotion.ACTION_BOTH_LOCK The motor is locked in both horizontal and vertical directions

Sample code:

```
LocateAbsoluteAngleHeadMotion locateAbsoluteAngleHeadMotion = new
LocateAbsoluteAngleHeadMotion(
    LocateAbsoluteAngleHeadMotion.ACTION_BOTH_LOCK,30,15
);
headMotionManager.doAbsoluteLocateMotion(locateAbsoluteAngleHeadM
otion);
```

3.4.5 Relative angle positioning operation

method signature:

public OperationResult

doRelativeLocateMotion(LocateRelativeAngleHeadMotion locateRelativeAngleHeadMotion)

Method Description:

Control the robot's head to rotate a certain angle relative to the current position, and decide whether to electrically

The machine is locked.

Parameter Description:

The **LocateRelativeAngleHeadMotion** class has five parameters: action, horizontalAngle

, verticalAngle, horizontalDirection and verticalDirection. action indicates the robot head lock mode, horizontalAngle indicates the horizontal relative angle of the robot head movement, verticalAngle indicates the vertical relative angle of the robot head movement, horizontalDirection indicates the horizontal movement direction of the robot head, and verticalDirection indicates the vertical movement direction of the robot head.

The following are the possible values that can be passed to the action parameter:

LocateRelativeAngleHeadMotion.ACTION_NO_LOCK The motor does not lock

LocateRelativeAngleHeadMotion.ACTION_HORIZONTAL_LOCK The motor is locked in the horizontal direction

LocateRelativeAngleHeadMotion.ACTION_VERTICAL_LOCK The motor is locked in the vertical direction

LocateRelativeAngleHeadMotion.ACTION_BOTH_LOCK The motor is locked in both horizontal and vertical directions

The following are the possible values passed by the horizontalDirection parameter:

LocateRelativeAngleHeadMotion.DIRECTION_LEFT horizontal movement to the left

LocateRelativeAngleHeadMotion.DIRECTION_RIGHT horizontal movement to the right

LocateRelativeAngleHeadMotion.DIRECTION_UP vertical upward movement

LocateRelativeAngleHeadMotion.DIRECTION_DOWN vertical downward movement

Sample code:

```
LocateRelativeAngleHeadMotion locateRelativeAngleHeadMotion = new
LocateRelativeAngleHeadMotion(
    LocateRelativeAngleHeadMotion.ACTION_BOTH_LOCK,(byte)
30,(byte) 15,
    LocateRelativeAngleHeadMotion.DIRECTION_LEFT
    ,LocateRelativeAngleHeadMotion.DIRECTION_UP
);
headMotionManager.doRelativeLocateMotion(locateRelativeAngleHeadM
otion);
```

3.5 Wing motion control

Access process:

1. Apply for WingMotionManager object, the application code is

```
WingMotionManager wingMotionManager =
```

```
(WingMotionManager)getManager(FuncConstant.WINGMOTION_MANAGER);
```

2. Use the applied wingMotionManager ob

3.5.1 No angle motion

method signature:

```
public OperationResult doNoAngleMotion(NoAngleWingMotion noAngleWingMotion)
```

Method Description:

Control the robot's wings to make non-angle movements, such as raising hands, returning wings to their original positions, etc.

Parameter Description:

The **NoAngleWingMotion** class has three parameters: action, part, and speed. action indicates the robot wing rotation mode, part indicates the position of the robot wing (left wing or right wing), and speed indicates the robot wing movement speed, with a value range of 1 to 10.

The following are the possible values that can be passed to the action parameter:

NoAngleWingMotion.ACTION_UP The wings turn upwards

NoAngleWingMotion.ACTION_DOWN The wings turn downward

NoAngleWingMotion.ACTION_STOP The wings stop moving

NoAngleWingMotion.ACTION_RESET The wings return to the default position (ie, the wings are vertically downward).

The following are possible values for the part parameter:

NoAngleWingMotion.PART_LEFT Left wing operation

NoAngleWingMotion.PART_RIGHT Right wing operation

NoAngleWingMotion.PART_BOTH Left and right wings operate together

Sample code:

```
NoAngleWingMotion noAngleWingMotion = new  
NoAngleWingMotion(NoAngleWingMotion.PART_BOTH,  
                    5, NoAngleWingMotion.ACTION_UP);  
wingMotionManager.doNoAngleMotion(noAngleWingMotion);
```

3.5.2 Relative angle motion

method signature:

```
public OperationResult doRelativeAngleMotion(RelativeAngleWingMotion relativeAngleWingMotion)
```

Method Description:

Control the robot's wings to rotate a certain angle in the specified direction relative to the current position.

Parameter Description:

The **RelativeAngleWingMotion** class has four parameters: action, part, speed, and angle. action indicates the robot's wing rotation mode, part indicates the position of the robot's wing (left wing or right wing), speed indicates the robot's wing movement speed, ranging from 1 to 8, and angle indicates the relative angle of the robot's wing rotation, ranging from 0 to 270, and the angle gradually increases in the counterclockwise direction.

The following are the possible values that can be passed to the action parameter:

RelativeAngleWingMotion.ACTION_UP The wings turn upward

RelativeAngleWingMotion.ACTION_DOWN The wings turn downward

The following are possible values for the part parameter:

RelativeAngleWingMotion.PART_LEFT Left wing operation

RelativeAngleWingMotion.PART_RIGHT Right wing operation

RelativeAngleWingMotion.PART_BOTH Left and right wings operate together

Sample code:

```
RelativeAngleWingMotion relativeAngleWingMotion = new
RelativeAngleWingMotion( RelativeAngleWingMotion.PART_LEFT,5,

RelativeAngleWingMotion.ACTION_UP,
    30

);
wingMotionManager.doRelativeAngleMotion(relativeAngleWingMotion);
```

[3.5.3 Absolute angle movement](#)

method signature:

public OperationResult doAbsoluteAngleMotion(AbsoluteAngleWingMotion absoluteAngleWingMotion)

Method Description:

Control the robot's wings to rotate to the specified angle.

Parameter Description:

The **AbsoluteAngleWingMotion** class has three parameters: speed, part, and angle. speed indicates the speed of the robot's wings, with a value range of 1 to 8, part indicates the position of the robot's wings (left or right), and angle indicates the absolute angle of the robot's wings, with a value range of 0 to 270, and the angle increases gradually in the counterclockwise direction.

The following are possible values for the part parameter:

AbsoluteAngleWingMotion.PART_LEFT Left wing operation

AbsoluteAngleWingMotion.PART_RIGHT Right wing operation

AbsoluteAngleWingMotion.PART_BOTH Operate both left and right wings

Sample code:

```
AbsoluteAngleWingMotion absoluteAngleWingMotion = new
AbsoluteAngleWingMotion(
    AbsoluteAngleWingMotion.PART_LEFT,5,0
);
wingMotionManager.doAbsoluteAngleMotion(absoluteAngleWingMotion);
```

3.6 Wheel Motion Control

Access process:

1. Apply for WheelMotionManager object, the application code is

```
WheelMotionManager wheelMotionManager= (WheelMotionManager) getUnit
Manager(FuncConstant.WHEELMOTION_MANAGER); 2. Use the applied
wheelMotionManager object to call the corresponding method for control.
```

3.6.1 No angle motion

method signature:

public OperationResult doNoAngleMotion(NoAngleWheelMotion noAngleWheelMotion)

Method Description:

Control the robot's wheels to make non-angle movements, such as forward, backward, left, etc.

Parameter Description:

The **NoAngleWheelMotion** class has three parameters: speed, action, and duration. Speed indicates the robot wheel movement speed, with a value range of 1 to 10, action indicates the robot wheel movement mode, and duration indicates the robot wheel movement time, in units of 100ms. When duration is 0, it means that the robot will remain in motion until it receives a stop command.

The following are the possible values that can be passed to the action parameter:

NoAngleWheelMotion.ACTION_FORWARD forward

NoAngleWheelMotion.ACTION_LEFT turns left in place

NoAngleWheelMotion.ACTION_RIGHT Turn right in place

NoAngleWheelMotion.ACTION_TURN_LEFT Turn left

NoAngleWheelMotion.ACTION_TURN_RIGHT Turn right

NoAngleWheelMotion.ACTION_STOP_TURN Stop turning

NoAngleWheelMotion.ACTION_STOP_RUN Stop walking

Sample code:

```
NoAngleWheelMotion noAngleWheelMotion2 = new NoAngleWheelMotion(  
    NoAngleWheelMotion.ACTION_FORWARD_RUN, 5, 3000  
);  
wheelMotionManager.doNoAngleMotion(noAngleWheelMotion2);
```

3.6.2 Relative angle motion

method signature:

public OperationResult doRelativeAngleMotion(RelativeAngleWheelMotion relativeAngleWheelMotion)

Method Description:

Control the robot's wheels to rotate a certain angle in the specified direction relative to the current position.

Parameter Description:

The **RelativeAngleWheelMotion** class has three parameters: speed, action, and angle. speed indicates the robot wheel movement speed, with a value range of 1 to 10, action indicates the robot wheel movement mode, and angle indicates the relative angle of the robot wheel rotation.

The following are the possible values that can be passed to the action parameter:

RelativeAngleWheelMotion.TURN_LEFT turns left

RelativeAngleWheelMotion.TURN_RIGHT Turn right

RelativeAngleWheelMotion.TURN_STOP Stop rotation

Sample code:

```
RelativeAngleWheelMotion relativeAngleWheelMotion = new  
RelativeAngleWheelMotion(  
    RelativeAngleWheelMotion.TURN_LEFT, 5, 3000  
);
```

```
        RelativeAngleWheelMotion.TURN_LEFT, 5,90
    );
wheelMotionManager.doRelativeAngleMotion(relativeAngleWheelMotion
);
```

3.6.3 Distance motion

method signature:

```
public OperationResult doDistanceMotion(DistanceWheelMotion distanceWheelMotion)
```

Method Description:

Control the robot to move a specified distance in a specified direction.

Parameter Description:

The **DistanceWheelMotion** class has three parameters: speed, action, and distance. speed indicates the robot wheel movement speed, with a value range of 1 to 10, action indicates the robot wheel movement mode, and distance indicates the robot movement distance, in cm.

The following are the possible values that can be passed to the action parameter:

DistanceWheelMotion.ACTION_FORWARD_RUN forward

DistanceWheelMotion.ACTION_STOP_RUN Stop walking

Sample code:

```
DistanceWheelMotion distanceWheelMotion = new DistanceWheelMotion(
    DistanceWheelMotion.ACTION_FORWARD_RUN, 5,100
);
wheelMotionManager.doDistanceMotion(distanceWheelMotion);
```

3.6.4 Wheel motion status callback method

description:

After the robot wheels move, the wheel movement status will be called back

Parameter Description:

The callback status s description, s = "0" means the motion is stopped, non-"0" means the motion state

Sample code:

```
wheelMotionManager.setWheelMotionListener(new  
WheelMotionManager.WheelMotionListener() {  
    @Override  
    public void onWheelStatus(String s) {  
        Log.i("Cris", "onWheelStatus: s="+s);  
    }  
});
```

3.7 System Management

Access conditions: Some functions can only be accessed by apps that have been signed by the system. The functions that require a system signature for access are marked below.

Access process:

1. Apply for SystemManager object, the application code is

```
SystemManager systemManager= (SystemManager)getUnitManager(FuncCon  
stant. SYSTEM_MANAGER); 2.
```

Use the applied `systemManager` object to call the corresponding method for control.

3.7.1 Get device ID method

signature:

```
public String getDeviceId();
```

Method Description:

Get the device ID of the current robot.

Sample code:

```
systemManager.getDeviceId();
```

3.7.2 Return to the screen saver animation

interface method signature:

```
public OperationResult doHomeAction()
```

Method Description:

Let the robot display jump to the desktop screen saver animation interface.

Return value description:

OperationResult's ErrorCode may return ErrorCode.FAIL_APP_HAS_LOCKED error,

Indicates that an app is currently locked and you cannot return to the screen saver animation interface.

Sample code:

```
systemManager.doHomeAction();
```

3.7.3 Expression control

method signature:

```
public OperationResult showEmotion(EmotionsType emotion)
```

Method Description:

Make the robot display the specified expression. This function will only take effect when the current display page is a face interface.

Parameter Description:

The following are the possible values passed by the emotion parameter:

EmotionsType.ARROGANCE Arrogance

EmotionsType.SURPRISE

EmotionsType.WHISTLE Whistling

EmotionsType.LAUGHTER

EmotionsType.GOODBYE Goodbye

EmotionsType.SHY

EmotionsType.SWEAT

EmotionsType.SNICKER Smirk

EmotionsType.PICKNOSE

EmotionsType.CRY tears

EmotionsType.ABUSE swearing

EmotionsType.ANGRY

EmotionsType.KISS

EmotionsType.SLEEP

EmotionsType.SMILE

EmotionsType.GRIEVANCE

EmotionsType.QUESTION

EmotionsType.FAINT

EmotionsType.PRISELike

EmotionsType.NORMALDefault

Sample code:

```
systemManager.showEmotion(EmotionsType. PRISE);
```

3.7.4 Get battery power

method signature:

```
public int getBatteryValue()
```

Method Description:

Used to obtain the current battery power of the robot.

Return value description:

The result field of OperationResult returns the battery power, and the power data is of int type.

3.7.5 Get battery status

method signature:

```
public int getBatteryStatus()
```

Method Description:

Used to obtain the current battery charging status of the robot.

Return value description:

The result field of OperationResult returns the charging status of the robot. There are three types of result:

SystemManager.STATUS_NORMAL Not charging

SystemManager.STATUS_CHARGE_PILE Using the charging station to charge the robot

SystemManager.SATUS_CHARGE_LINE Using the power cable to charge the robot

3.7.6 Monitoring security home alarm

method signature:

```
public void setOnIDarlingListener(IDarlingListener iDarlingListener)
```

Method Description:

If the security home has enabled the deployment strategy, this callback method will be called when the security home alarm event is triggered.

Parameter Description:

The security home alarm callback implements the IDarlingListener interface. In this interface, the following methods need to be implemented:

```
void onAlarm(int type);
```

type indicates the security alarm mode. When the type value is 1, it means that the occlusion alarm is currently triggered. When it is 2,

Indicates that an intrusion alarm is triggered. When it is 3, it indicates that a cross-border alarm is triggered.

[3.7.7 Get the master version number](#)

method signature:

```
public String getMainServiceVersion()
```

Method Description:

Get the main control program version number of the current system.

[3.7.8 Show/hide system floating button method](#)

signature:

```
public OperationResult switchFloatBar(boolean isShow,String className)
```

Method Description:

Show/hide the floating button of the current system. This method can only be called in Activity, and will not take effect if called in Service.

Parameter Description:

isShow Whether to display the system's floating button

className is the full class name of the current Activity, which can be obtained through the getClass().getName() method.

[3.7.9 Show/hide system floating button \(to avoid flashing when jumping to Activity in APP\) method signature:](#)

```
public OperationResult switchFloatBarInApplication(boolean isShow)
```

Method Description:

Show/hide the floating button of the current system.

Parameter Description:

isShow Whether to display the system's floating button

3.7.10 Query robot type method

signature:

```
public int getRobotType();
```

Method Description:

Get the robot type

Method return description:

SystemManager.ROBOT_TYPE_B; (Type B, elf)

SystemManager.ROBOT_TYPE_D; (Type D, King Kong)

SystemManager.ROBOT_TYPE_DESKTOP; (desktop robot, mini elf)

3.8 Multimedia Management

Access process:

1. Apply for **HDCameraManager** object, the application code is

```
HDCameraManager hdCameraManager =  
(HDCameraManager)getUnitManager(FuncConstant.HDCAMERA_MANAGER);
```

2. Use the applied **hdCameraManager** object and call the corresponding method for control.

3.8.1 Get audio and video stream callback

method signature:

```
public void setMediaListener(MediaListener mediaListener)
```

Method Description:

Audio and video streaming data is obtained through the robot's high-definition camera and microphone.

Precautions:

1. The microphone of the HD camera does not support echo cancellation, so the local recording and playback

Sometimes there will be very sharp noise, so you need to be careful when using audio.

Parameter Description:

The callback for getting audio and video streams is implemented by the `MediaStreamListener` interface. In this interface, the following methods need to be implemented:

```
void getVideoStream(byte[] data);  
  
void getAudioStream(byte[] data);
```

The above two methods correspond to the video stream data callback and audio stream data callback methods respectively, and the obtained audio and video streams
For details on the processing, please refer to the code in the provided Demo.

3.8.2 Open media stream

method signature:

```
public OperationResult openStream(StreamOption streamOption)
```

Method Description:

The method to open the media stream. Only after calling this method can the callback data of the audio and video stream be obtained.

Parameter Description:

StreamOption has the following parameters:

type indicates the decoding method used by the video. The possible value range is:

StreamOption.TYPE_HARDWARE_DECORD hard decoding (default value)

StreamOption.TYPE_SOFTWARE_DECORD software decoding

isJustIframe indicates whether to return only I frame (please note that this is the English letter I, not the number 1) data. The default value is
false.

channel indicates the stream format of the returned video. The possible value range is:

StreamOption.MAIN_STREAM main stream, the returned video resolution is **1280*720 (default value)**

StreamOption.SUB_STREAM sub-stream, the returned video resolution is **640*480**

The example code is as follows:

```
StreamOption streamOption = new StreamOption();  
streamOption.setChannel(StreamOption.MAIN_STREAM );  
streamOption.setType(StreamOption.TYPE_HARDWARE_DECORD);  
  
streamOption.setChannel(StreamOption.SUB_STREAM).setDecodType(StreamOption.TYPE_SOFTWARE_DECORD).setJustIframe(false);  
OperationResult operationResult = hdCameraManager.openStream(streamOption);  
int result = Integer.parseInt(operationResult.getResult());  
if (result != -1){
```

```
handleList.add(result);
```

} Now, when opening a video stream, a maximum of two video streams are

allowed to be pulled. The `hdCameraManager.openStream(streamOption)` method will return an `OperationResult` object when opening it.

According to the `getResult()` method of `operationResult`, the instructions for opening the stream are obtained. If the opening is successful, a string greater than 0 is returned, which needs to be converted to an `int` type. The purpose of the instructions is to pass in the instructions when closing the stream.

3.8.3 Close media stream

method signature:

```
public OperationResult closeStream(int handle)
```

Method Description:

Close the media stream. If you have called the `openStream` method, please remember to destroy it in the interface.

Be sure to call this method to close the media stream, otherwise it will cause unexpected risks such as memory leaks.

Example Code

`hdCameraManager.closeStream(handle)`), where `handle` represents the pointer returned when opening the stream

3.8.4 Get the face recognition callback

method signature:

```
public void setMediaListener(MediaListener mediaListener)
```

Method Description:

This callback method is triggered when the robot recognizes a face in front of the robot. If the current user has entered information in the Family Member App, if the robot recognizes the corresponding user, it will return the user's information at the same time. Please note that entering a user in the Family Member App does not mean that the robot can definitely recognize the user. The recognition accuracy will also be affected by factors such as the current ambient light and the angle of the face. This method will not be called back in an environment without **a network**.

Parameter Description:

The `FaceRecognizeListener` interface is used to obtain the face recognition callback. In this interface, the following methods need to be implemented:

```
void recognizeResult(List<FaceRecognizeBean> faceRecognizeBean);
```

`FaceRecognizeBean` contains the position of the recognized face image on the screen and the user's family member entry

The relevant personal information (if any), which includes the following parameters:

private int w (width of the face recognition image, currently the default is **1280**)

private String birthday (identified user's birthday information, the user information needs to have been entered in the family members)

private double bottom (the percentage of the screen occupied by the bottom position of the face)

private String gender (the gender information of the recognized user, the user information needs to have been entered among the family members) **private double** left (the percentage of the screen occupied by the leftmost position of the face) **private String** qlink (reserved field) **private double** right (the percentage of the screen occupied by the rightmost position of the face) **private String** user (the name information of the recognized user, the user information needs to have been entered among the family members) **private int** h (the height of the face recognition image, the current default is **720**) **private double** top (the percentage of the screen occupied by the top position of the face)

Sample code:

```
hdCameraManager.setMediaListener(new FaceRecognizeListener() {  
    @Override  
    public void recognizeResult(List<FaceRecognizeBean>  
faceRecognizeBean) {  
        Log.i("info", "Acquired face recognition data");  
    }  
});
```

3.8.5 Method signature for capturing images

from a video stream :

public Bitmap getVideoImage()

Method Description:

If you need to grab an image from the robot's video stream, you can call this method. This method is used together with method 3.9.4 to grab a human face.

Sample code:

```
hdCameraManager.setMediaListener(new FaceRecognizeListener() {  
    @Override  
    public void recognizeResult(FaceRecognizeBean faceRecognizeBean)  
{  
        Bitmap bitmap = mediaManager.getVideoImage();  
        if(bitmap != null){  
            Log.i("info", "Get the face recognition image");  
        }  
    }  
});
```

```
    }  
    });
```

3.9 Projector Control

Access process:

1. Apply for a ProjectorManager object. The application code is

```
ProjectorManager projectorManager = (ProjectorManager)getUnitManager(FuncConstant.PROJECTOR_MANAGER);
```

2. Use the applied projectorManager object to call the corresponding method for control.

3.9.1 Switch projector method

signature:

```
public OperationResult switchProjector(boolean isOpen)
```

Method Description:

Used to control the opening and closing of the projector. Please note that each opening and closing operation of the projector needs to be separated by at least 12 seconds, otherwise the command will not be executed.

Parameter Description:

isOpen true means turn on the projector, otherwise turn it off

Sample code:

```
projectorManager.switchProjector(true);
```

3.9.2 Set the projector image method

signature:

```
public OperationResult setMirror(int value)
```

Method Description:

Used to set up the projector mirror.

Parameter Description:

The value range is from 0 to 3, and the specific mirroring modes are as follows:

ProjectorManager.MIRROR_CLOSE 0 No flip, default mode

ProjectorManager.MIRROR_LR 1 Flip left and right

ProjectorManager.MIRROR_UD 2 flip upside down

ProjectorManager.MIRROR_ALL 3 Flip up and down, left and right

Sample code:

```
projectorManager.setMirror(ProjectorManager.MIRROR_ALL);
```

3.9.3 Set the projector trapezoidal horizontal correction value

method signature:

```
public OperationResult setTrapezoidH(int value)
```

Method Description:

Used to set the projector's horizontal keystone correction value.

Parameter Description:

The valid value range of **Value** is -30 ~ 30

Sample code:

```
projectorManager.setTrapezoidH(10);
```

3.9.4 Set the projector keystone vertical correction value

method signature:

```
public OperationResult setTrapezoidV(int value)
```

Method Description:

Used to set the projector's vertical keystone correction value.

Parameter Description:

The valid value range of **Value** is -20 ~ 30

Sample code:

```
projectorManager.setTrapezoidV(10);
```

3.9.5 Set Projector Contrast Method

signature:

```
public OperationResult setContrast(int value)
```

Method Description:

Used to set the contrast of the projector image.

Parameter Description:

The valid value range of **Value** is -15 ~ 15

Sample code:

```
projectorMananger.setContrast(10);
```

[3.9.6 Set the projector brightness](#)

method signature:

```
public OperationResult setBright(int value)
```

Method Description:

Used to set the brightness of the projector screen.

Parameter Description:

The valid value range of **Value** is -31 ~ 31

Sample code:

```
projectorMananger.setBright(10);
```

[3.9.7 Set the projector color value](#)

method signature:

```
public OperationResult setColor(int value)
```

Method Description:

Used to set the color value of the projector image.

Parameter Description:

The valid value range of **Value** is -15 ~ 15

Sample code:

```
projectorMananger.setColor(10);
```

3.9.8 Set the projector screen saturation method

signature:

```
public OperationResult setSaturation(int value)
```

Method Description:

Used to set the saturation of the projector image.

Parameter Description:

The valid value range of **Value** is -15 ~ 15

Sample code:

```
projectorMananger.setSaturation(10);
```

3.9.9 Set the projector image sharpness

method signature:

```
public OperationResult setAcuity(int value)
```

Method Description:

Used to set the sharpness of the projector image.

Parameter Description:

The valid value range of **Value** is 0~6

Sample code:

```
projectorMananger.setAcuity(4);
```

3.9.10 Projector expert mode optical axis adjustment

method signature:

```
public OperationResult setExpertAxis(int value)
```

Method Description:

Used to set the optical axis of the projector expert mode.

Parameter Description:

Value 1 Open setting 2 Increase value 3 Decrease value 4 Exit setting without saving 5 Save and exit setting

Sample code:

```
projectorManager.setExpertAxis(1);
```

3.9.11 Projector expert mode phase adjustment

method signature:

```
public OperationResult setExpertPhase(int value)
```

Method Description:

Used to set the phase of the projector expert mode.

Parameter Description:

Value 1 Open setting 2 Increase value 3 Decrease value 4 Exit setting without saving 5 Save and exit setting

Sample code:

```
projectorManager.setExpertPhase(1);
```

3.9.12 Set projector mode method

signature:

```
public OperationResult setMode(int mode)
```

Method Description:

Used to set the projection mode of the projector.

Parameter Description:

Value **ProjectorManager.MODE_WALL** 1 Wall projection mode

ProjectorManager.MODE_CEILING 2 Ceiling projection mode

Sample code:

```
projectorManager.setMode(ProjectorManager.MODE_CEILING);
```

3.9.13 Query projector data and status method

signature:

```
public OperationResult queryConfig(String configName)
```

Method Description:

Used to query whether the current projector is on or off, as well as the related sharpness, contrast, color

The numerical value of the setting.

The possible values for configName are:

ProjectorManager.CONFIG_SWITCH	Query switch status
ProjectorManager.CONFIG_TRAPEZOIDH	Query the horizontal trapezoidal value
ProjectorManager.CONFIG_TRAPEZOIDV	Query the vertical trapezoid value
ProjectorManager.CONFIG_CONTRAST	query contrast value
ProjectorManager.CONFIG_BRIGHT	Query brightness value
ProjectorManager.CONFIG_COLOR	Query chromaticity value
ProjectorManager.CONFIG_SATURATION	Query the saturation value
ProjectorManager.CONFIG_ACUITY	Query sharpness value
ProjectorManager.CONFIG_MODE	Query Mode
ProjectorManager.CONFIG_MIRROR	Query image

Sample code:

```
OperationResult operationResult =
projectorManager.queryConfig(ProjectorManager.CONFIG_SWITCH);

if(operationResult.getResult().equals("1")){

    Log.e("info","The projector is turned on")

}
```

3.9.14 Projector interface callback

Method signature:

```
public void setOnProjectorListener(OnProjectorListener projectorListener)
```

Method Description:

Projector-related monitoring interface, used to monitor errors that occur during projector expert mode adjustment.

Sample code:

```
projectorManager.setOnProjectorListener(new
    ProjectorManager.ProjectorListener() {

        @Override
```

```
        public void expertModeError(ProjectorExpertMode
projectorExpertMode){

        //TODO Projector expert mode adjustment error callback

    }

});
```

3.10 Modular motion function control

Access process:

1. Apply for ModularMotionManager object, the application code is

```
ModularMotionManager modularMotionManager= (ModularMotionManager)get UnitManager(FuncConstant.MODULARMOTION_MANAGER);
```

2. Use the applied modularMotionManager object to call the corresponding method for control.

3.10.1 Turn on/off free walking method

signature:

```
public OperationResult switchWander(boolean isOpen)
```

Method Description:

Control the robot's free walking function on and off.

Return value description:

The ErrorCode of OperationResult may have the following return values:

ErrorCode.FAIL_MOTION_LOCKED The robot is currently executing other motion-related functions.

The motion control is locked and cannot respond to commands.

ErrorCode.FAIL_NO_PERMISSION The current user group does not have permission to operate this function

ErrorCode.FAIL_IS_CHARGE The robot is currently charging and the corresponding function cannot be opened.

3.10.2 Turn on/off automatic charging method

signature:

```
public OperationResult switchCharge(boolean isOpen)
```

Method Description:

Control the robot's automatic charging function on and off.

Return value description:

The ErrorCode of OperationResult may have the following return values:

ErrorCode.FAIL_MOTION_LOCKED The robot is currently executing other motion-related functions.

The motion control is locked and cannot respond to commands.

ErrorCode.FAIL_NO_PERMISSION The current user group does not have permission to operate this function

ErrorCode.FAIL_IS_CHARGE The robot is currently charging and the corresponding function cannot be opened.

ErrorCode.FAIL_NO_CHARGE_PILE The robot did not find the charging pile. Please confirm whether the charging pile is plugged in or has been added to the smart circle.

3.10.3 Get the free walking switch status method

signature:

```
public OperationResult getWanderStatus()
```

Method Description:

Determine whether the robot's free walking function is turned on.

Return value description:

The getResult() method of OperationResult will return the query result. If the returned value is "1", it means

"0" indicates the wandering switch is turned on, and "0" indicates it is turned off.

Sample code:

```
OperationResult operationReusult =  
modularMotionManager.getWanderStatus();  
if(operationResult.getResult().equals("1")){ //TODO wandering  
function is turned on}
```

3.10.4 Get the automatic charging switch status method

signature:

```
public OperationResult getAutoChargeStatus()
```

Method Description:

Determine whether the robot's automatic charging function is turned on.

Return value description:

The getResult() method of OperationResult will return the query result. If the returned value is "1", it means

"0" indicates that the automatic charging switch is turned on, and "0" indicates that it is turned off.

Sample code:

```
OperationResult operationResult = modularMotionManager.  
getAutoChargeStatus();  
if(operationResult.getResult().equals("1")){ //TODO automatic  
charging function is turned on }
```

3.11 Preprocessing Directives

Preprocessing instructions are instructions that are executed immediately when the Activity or Service component establishes a connection with the main control. Preprocessing instructions are configured in the AndroidManifest.xml file in the form of <meta-data> tags, which are configured under the <application>, <activity>, and <service> tags, respectively, with different scopes. The following is a schematic diagram of preprocessing instructions in the AndroidManifest.xml file:

```
<application android:allowBackup="true" android:icon="@mipmap/ic_launcher"  
  android:label="QihanOpenSDK Demo" android:supportsRtl="true" android:theme:  
  >  
  <meta-data android:name="SPEECH_MODE" android:value="1"/> ←  
  <activity android:name=".MainActivity">  
    <intent-filter>  
      <action android:name="android.intent.action.MAIN" />  
      <category android:name="android.intent.category.LAUNCHER" />  
    </intent-filter>  
    <meta-data android:name="FORBID_PIR" android:value="true"/> ←  
  </activity>  
  <activity android:name=".VideoActivity">  
    android:hardwareAccelerated="true">  
    <meta-data android:name="config_record" android:value="false"/> ←  
  </activity>  
  
  <activity android:name=".FaceRecognizeActivity"/>  
  <service android:name=".MainService">  
    <meta-data android:name="RECOGNIZE_MODE" android:value="1"/> ←  
  </service>  
</application>
```

The code snippet indicated by the red arrow is the preprocessing directive.

The preprocessing directive configured under the <application> tag is scoped to all activities and services under the current application. It is a global attribute, which is equivalent to configuring the same preprocessing directive for each activity or service. The preprocessing directive configured under the <activity> tag will take effect when the current activity is visible and will become invalid when the onStop method of the current activity is called. The preprocessing directive configured under the <service> tag will take effect when the Service is created and will become invalid when the onDestroy method of the Service is called.

Preprocessing instructions under <application> will be overwritten by the same preprocessing instructions under <activity> or <service>. If you need to use preprocessing instructions under <application> or <service>, be careful because Services can

Due to the background running characteristics, once the preprocessing instruction is configured, it will always take effect due to the survival of the Service, which will affect the interaction logic of the robot itself and the interaction of other third-party applications. Therefore, if it is not necessary, please avoid using preprocessing instructions under <application> or <service>.

Note: Due to the limitations of the working mechanism of preprocessing instructions, when reinstalling or uninstalling the app in the R&D system, the robot may not work properly due to the influence of the preprocessing instructions. This phenomenon is only related to the reinstallation and uninstallation of the app and will not affect the normal use of the app after it is put on the shelf.

3.11.1 Recording switch

method signature:

```
<meta-data android:name="CONFIG_RECORD" android:value="true"/>
```

Method Description:

If the robot needs to use the recording function, this command must be configured. If this command is successful, the robot's ear will light up blue. Please be sure to configure according to the above instructions, otherwise it will cause the main control to restart and the connection between the app and the main control will be disconnected.

3.11.2 Identification mode

method signature:

```
<meta-data android:name="RECOGNIZE_MODE" android:value="1"/>
```

Method Description:

Set the robot speech recognition mode. Currently only the truncation mode is supported. For details, please refer to Section 3.2.6.
bright.

3.11.3 Voice mode method

signature:

```
<meta-data android:name="SPEECH_MODE" android:value="1"/>
```

Method Description:

The robot has two built-in speech recognition modes: 1. After the robot is awakened, if there is no speech for more than 10 seconds, When it detects someone talking, it will automatically enter the sleep state. This mode is the default mode of the robot. Second, after the robot is awakened, if no one is detected talking within a short period of time (about 2 to 3 seconds), it will immediately enter the sleep state. When the robot recognizes the words spoken by the user, it will also enter the sleep state. In this mode, each time the user completes a conversation with the robot, they need to wake up the robot again to continue communicating with the robot.

When android:value="1", the robot will enter the above-mentioned mode 2.

3.11.4 Touch response switch

method signature:

```
<meta-data android:name="FORBID_TOUCH" android:value="true"/>
```

Method Description:

After configuring this preprocessing instruction, the user can still listen to touch events, but the robot will not respond to touch events.

The robot can respond to any event, including default behaviors such as waking up the robot on touch.

3.11.5 PIR response switch

method signature:

```
<meta-data android:name="FORBID_PIR" android:value="true"/>
```

Method Description:

After configuring this pre-processing instruction, the user can still monitor the PIR triggered event, but the robot will not

The PIR trigger event is executed according to the default strategy (the default strategy means that when the PIR on the back of the robot is triggered, the robot will rotate 180 degrees).

3.11.6 Voice wake-up response switch

method signature:

```
<meta-data android:name="FORBID_WAKE_RESPONSE" android:value="true"/>
```

Method Description:

After configuring this preprocessing instruction, the robot will not make any default response behaviors when it is voice-awakened, except that the ear light turns green.

3.11.7 Misidentification mode

method signature:

```
<meta-data android:name="MISTAKENLY_IDENTIFIED" android:value="true"/>
```

Method Description:

The master will filter the recognition of one word by default. After setting this preprocessing instruction, the master will no longer filter the recognition of one word. Identification, such as "yes", "no", etc. This mode is best used in a quiet environment, and noisy environments will always misidentify.

3.11.8 Anti-accidental killing mode (the app will be killed when playing music in the background)

method signature:

```
<meta-data android:name="forbid_stop_music" android:value="true"/>
```

Method Description:

In previous versions, apps that play music in the background were killed by default. For example, if the current page is playing music and then jumps to another page while the music is still playing, the app will be killed. Now a pre-processing mode is added. If this mode is configured, the app will not be killed again. However, the app must control the life cycle of the playback to avoid affecting the recognition effect.

[3.11.9 Disable semantic request mode \(only applicable to foreign versions\)](#) Method signature:

```
<meta-data android:name="forbid_request_semantics"android:value="1"/>
```

Method Description:

The user configures the meta-data on the Activity corresponding to the AndroidManifest, and the main service will read this configuration. If the configuration is 1, the main service will not make a network request, and the client can make its own semantic request based on the text recognition returned by the main service.

[3.12 Command words and semantics](#)

[3.12.1 Customizing global command words](#)

Users can control the robot through certain specific words or short sentences, which we call command words. For example, we can use the command word "turn on the light" to control the robot to turn on the white light on its head, and use the command word "play movie" to control the robot to open the movie app and play the movie. These command words are global and always take effect no matter what interface the system is in.

The SDK supports developers to customize global command words to call up the Activity or Service specified by the developer.

When using custom global command words, developers need to understand the following:

1. Global command words support offline recognition, and command words can take effect even when not connected to the Internet.
2. Each app supports 10 custom command words, and the entire system supports less than 100 custom global commands
Command words exceeding this limit will be automatically ignored.
3. The parsing order of command words is uncertain, which means that if your app and other developers' app definitions
If you use the same command word, it is likely that your command word will not take effect.
4. The global command words will be preloaded when the system is started for the first time. If the developer makes changes to the global command words during development, it is necessary to restart the robot or restart the main service program to make them take effect.

The development process of custom global command words is as follows:

1> Add the following configuration in the AndroidManifest.xml file:

```
<provider  
    android:authorities="<your app package name>.grammar"
```

```
android:name="com.sanbot.opensdk.utils.GrammarProvider" android:exported="true"/
```

> Note that the content in the above

brackets should be replaced with the corresponding app package name.

2> Create a new grammar file in the assets directory and create a new folder named

The file is globalgrammar.xml.

3> Write the grammar of global command words in globalgrammar.xml.

```
<Grammars>
  <group>
    <command>
      <item>Play video</item>
      <item>Open video</item>
    </command>
    <type>activity</type>
    <class>com.sanbot.librarydemo.VideoActivity</class>
  </group>
  <group>
    <command>
      <item>Start the service</item>
    </command>
    <type>service</type>
    <class>com.sanbot.librarydemo.MainService</class>
  </group>
</Grammars>
```

The above is an example of the global command syntax. When the user says "play video" or "open video" to the robot, the robot will automatically open the com.sanbot.librarydemo.VideoActivity page. The following will introduce the xml tags in the example in detail.

<Grammars>——The root tag of grammar.

<group>——<group> defines a group of command words and the definition of the command words to start the Activity or Service behavior.

righteous.

<command>——<command> contains a group of command words with the same behavior.

<item>——Each <item> contains a command word.

<type>——The object started when the command word is triggered. The optional values are activity and service.

<class>——The class name of the class that needs to be started. Note that the complete class name information must be filled in.

4> For the activity or service that needs to be started, add the following configuration in the androidmanifest.xml file:

```
android:exported="true"
```

```
<activity android:name=".VideoActivity"
    android:hardwareAccelerated="true"
    android:exported="true">
    <meta-data android:name="config_record" android:value="false"/>
</activity>
```

The triggered global command word will be passed to the activity or service through Intent. If the developer needs to obtain the triggered command word, he can refer to the following code

```
public class TestActivity extends BindBaseActivity {

    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState); String
        data = getIntent().getStringExtra("content");
    }
    .....
}
```

3.12.2 Customized semantics In

the speech recognition callback interface (refer to Section 3.2.6), the interface returns not only what the user said to the robot, but also the semantic parsing results that the robot can recognize. However, this kind of semantic parsing support is imperfect and often cannot meet the developer's needs for semantics in various specific scenarios. Therefore, the SDK provides developers with a way to customize semantics.

When using custom semantics, developers need to be aware of the following:

1. Custom semantics can generally only be used when connected to the Internet.
2. Each app supports 20 customized semantics. Semantics exceeding this limit will be automatically ignored.
3. Custom semantics will only take effect when the user's app is in the foreground.
Monitoring with custom semantics is unreliable behavior.
4. Custom semantics will be preloaded when the system is started for the first time.

If the definition is modified, you need to restart the robot or restart the main service program to make it take effect.

The development process of custom global command words is as follows:

1>Add the following configuration in the AndroidManifest.xml file:

```
<provider
    android:authorities="<your app package
    name>.grammar" android:name="com.sanbot.opensdk.utils.GrammarProvider"
    android:exported="true"/>
```

Please replace the content in the above brackets with the package name of the corresponding app.

2> Create a new grammar file in the assets directory, and create a file named localgrammar.xml in the newly created grammar folder.

3> Write the grammar of global command words in localgrammar.xml.

```
<Grammars>
  <group>
    <command>
      <item>Play video</item>
      <item>Open video</item>
    </command>
    <topic>qh_video</topic>
    <action>start</action>
  </group>
  <group>
    <command>
      <item>Pause video</item>
    </command>
    <topic>qh_video</topic> <action>pause</
    action>
  </group>
</Grammars>
```

The above is an example of the global command syntax. When the user says "play video" or "open video" to the robot, the robot will automatically open the com.sanbot.librarydemo.VideoActivity page. The following will introduce the xml tags in the example in detail.

<Grammars>——The root tag of grammar.

<group>——A group of semantics is defined in <group>.

<command>——<command> contains a group of semantics with the same topic and action.

<item>——Each <item> contains a phrase. When the robot recognizes this phrase, it will interpret it according to the user's own Define semantic rules for processing.

<topic>——Define the semantic topic. To avoid mixing with the built-in semantics of the system, the recommended naming method is "Company (Individual)

For details, please refer to the "Sambo Open Semantic Description Document".

<action>——Defines the semantic action. For details, please refer to the "Sambo Open Semantic Description Document".

3.13 Intelligent Peripheral Control

Access process:

1. Apply for ZigbeeManager object, the application code is

```
ZigbeeManager zigbeeManager = (ZigbeeManager)getUnitManager(FuncCons tant.ZIGBEE_MANAGER);
```

 2. Use the applied

zigbeeManager object to call the

corresponding method for control.

3.13.1 Get the whitelist

method signature:

```
public OperationResult getWhiteList()
```

Method Description:

Used to obtain the Zigbee whitelist. The specific result will be returned in the callback method.

3.13.2 Determine whether Zigbee is ready Method

signature:

```
public OperationResult getNotifyReadyStatus()
```

Method Description:

Check whether Zigbee is ready. Only when Zigbee is ready can other Zigbee operations be performed.

Return value description:

The getResult() method of OperationResult will return the query result. If the returned value is "1", it means

"0" indicates that Zigbee has been initialized successfully, and "0" indicates that Zigbee has not been initialized successfully.

Sample code:

```
OperationResult operationResult =zigbeeManager.  
getNotifyReadyStatus();  
if(operationResult.getResult().equals("1")){ //TODO NotifyReady  
  
}
```

3.13.3 Get device list method

signature:

```
public OperationResult getZigbeeList()
```

Method Description:

Used to obtain the Zigbee device list. The specific result will be returned in the callback method.

3.13.4 Add to whitelist method

signature:

```
public OperationResult addWhiteList(String command)
```

Method Description:

Used to add Zigbee devices to the whitelist.

3.13.5 Delete whitelist method

signature:

```
public OperationResult deleteWhiteList(String command)
```

Method Description:

Used to remove Zigbee devices from the whitelist.

3.13.6 Enable whitelist method

signature:

```
public OperationResult switchWhtieList(boolean isOpen)
```

Method Description:

Used to turn the whitelist function on or off.

Sample code:

```
zigbeeManager.switchWhtieList(true);
```

3.13.7 Set the time when the device is allowed to join

Method signature:

```
public OperationResult setAllowJoinTime(int time)
```

Method Description:

Set the time when the device is allowed to join the gateway. When set to 0, it means that the device is allowed to join at any time.

Gateway.

Parameter Description:

time : The time allowed to join, in seconds.

Sample code:

```
zigbeeManager.setAllowJoinTime(200);
```

3.13.8 Delete device

method signature:

```
public OperationResult deleteDevice(String command)
```

Method Description:

Remove the device from the gateway.

3.13.9 Clear list method

signature:

```
public OperationResult clearWhiteList()
```

Method Description:

Clear the Zigbee whitelist.

3.13.10 Send Byte command method

signature:

```
public OperationResult sendByteCommand(byte[] data)
```

Method Description:

Directly send byte array type data to the Zigbee module, generally used for communication between private devices.

3.13.11 Send String command method

signature:

```
public OperationResult sendCommand(String command)
```

Method Description:

Directly send String type commands to the Zigbee module.

3.13.12 Zigbee interface callback

method signature:

```
public void setZigbeeListener(ZigbeeListener zigbeeListener)
```

Method Description:

Zigbee-related monitoring interface, used to obtain whitelist data, monitor Zigbee data updates and status changes.

Sample code:

```
zigbeeManager.setZigbeeListener(new
ZigbeeManager.ZigbeeListener(){ @Override
    public void
        notifyWhiteList(String whiteListData) {
            //Get the whitelist data
        }

    @Override
    public void notifyStatusChange(String info) { //Device type data will
        be returned here}

    @Override
    public void notifyInfo(String info) { //Device list data will
        be returned here

    } });
```

3.14 Face tracking and proactive greeting

Access process:

1. Apply for a FaceTrackManager object. The application code is

```
FaceTrackManager faceTrackManager = (FaceTrackManager)getUnitManager (FuncConstant.FACETRACK_MANAGER);
```

2. Use the

applied faceTrackManager object to call the corresponding method

for control.

3.14.1 Face tracking switch method

signature:

```
public void switchFaceTrack(boolean isOn)
```

Method Description:

Directly send a boolean command to **FaceTrackManager**. **In a non-face interface, if you need to enable face tracking,**

The face tracking function can be turned on through this method. Similarly, if you need to turn off the face tracking function, just pass in false (if you do not manually turn off the face tracking function when exiting the interface, the system will automatically turn it off when the client is disconnected)

Sample code:

```
faceTrackManager.switchFaceTrack(true);
```

3.14.2 Face tracking status callback

method signature:

```
public void setFaceTrackListener
```

```
(new FaceTrackManager.FaceTrackListener() {})
```

Method Description:

Face tracking status callback, which will be called back only when the face tracking switch is turned on in the current page

Sample code:

```
faceTrackManager.setFaceTrackListener(new
FaceTrackManager.FaceTrackListener() {
    @Override
    public void faceTrackStart() {
        Log.i (TAG "faceTrackStart: ");
    }

    @Override
    public void faceTrackComplete() {
        Log.i (TAG "faceTrackComplete: ");
    }

    @Override
    public void faceTrackFail() {
        Log.i (TAG "faceTrackFail: ");
    }

    @Override
    public void faceTrackLost() {
        Log.i (TAG "faceTrackLost: ");
    }

    @Override
    public void startFindFace() {
        Log.i (TAG "startFindFace: ");
    }
});
```

1. The startFindFace() method triggers ir when a person stands in front of the robot. If no person is detected, a callback will be made.

When a method is called back, it means that the robot is trying to find a face by moving.

2. faceTrackStart() method indicates that a face has been detected and face tracking has started. This method will only return

Adjust once

3. The `faceTrackComplete()` method indicates that the face has been tracked and is in the center of the screen. The user can perform some logical operations such as face recognition. Note that this method may be triggered multiple times and may be triggered again after the user moves and tracks the face to the center again.
4. `faceTrackLost()` method description: If there is no face information within 10 seconds, the face has been lost. The method is only triggered after the `faceTrackStart` method is triggered.
5. The `faceTrackFail()` method is only triggered if no face is found within 10 seconds after the `startFindFace` method is triggered. If a face is found within 10 seconds, this method will not be triggered.

3.14.3 Active greeting switch method

signature:

```
public void switchSayHello(boolean isOn)
```

Method Description:

Directly send a boolean command to **FaceTrackManager**. If you want to use the active greeting function, please turn on the face tracking function first, because the active greeting will only be triggered when the face is tracked to the center of the screen.

Call function

Sample code:

```
faceTrackManager.switchSayHello(true);
```

true means on; false means off

3.14.4 Active greeting status callback method

signature:

```
public void setUsernameDetectListener(new  
FaceTrackManager.UserNameDetectListener() {})
```

Method Description:

The system will save the face information for 5 seconds. If there is no face information within 5 seconds, the face information will be automatically cleared. When the device is not in the speaking, waking up, or microphone-disabling state, the facial information will be displayed.

Sample code:

```
faceTrackManager.setUsernameDetectListener(new  
FaceTrackManager.UserNameDetectListener() { @Override  
  
    public void userName(String s) { "userName:  
        i (TAG $s"+s); Log.
```

```
    }  
});
```

Note that `s` may be an empty string.

3.15 Desktop Motion Control Protocol

Access process:

1. Apply for a `DesktopMotionManager` object. The application code is

```
DesktopMotionManager desktopManager = (DesktopMotionManager)getUnitManager(FuncConstant.DESKTOPMOTION_MANAGER);
```

2. Use the applied `desktopManager` object to call the corresponding method for control.

3.15.1 Stop motion method

signature:

```
public OperationResult doStop()
```

Method Description:

Send instructions to the background service directly through the `desktopManager` object. When the background service receives the instruction, it will stop multi-axis motion.

Sample code:

```
desktopManager.doStop();
```

3.15.2 Desktop motion reset method

signature:

```
public OperationResult doReset()
```

Method Description:

Send instructions to the background service directly through the `desktopManager` object. When the background service receives the instruction, it will reset.

Sample code:

```
desktopManager.doReset();
```

3.15.3 Desktop relative angle movement

method signature:

public OperationResult

doRelativeAngleMotion(RelativeAngleDesktopMotion relativeAngleDesktopMotion)

Method Description:

Send instructions to the background service directly through **the desktopManager** object. After receiving the instruction, the background service will perform the corresponding operation.

Speed can be positive or negative, but angle cannot be positive or negative

Vertical angle control range: 0-45

Horizontal angle control range: 0-160

Roll angle control range: 0-160

Front and rear angle control range: 0-45

Sample code:

```
RelativeAngleDesktopMotion relativeAngleDesktopMotion=new RelativeAngleDesktopMotion();

relativeAngleDesktopMotion.setForwardDegree((short) 10); //The required forward and backward movement angle is 10

relativeAngleDesktopMotion.setForwardSpeed((short) 10); // Move backward, speed control 1-60, positive number moves backward, negative number moves forward.

Speed controls the forward and backward movement direction

relativeAngleDesktopMotion.setHorizontalDegree((short) 10); //Horizontal motion angle 10

relativeAngleDesktopMotion.setHorizontalTurnSpeed((short) 10); // Speed control 1-60 moves to the left, positive number moves to the left, negative number moves to the right

the left and right movement direction is controlled by positive and negative speed

relativeAngleDesktopMotion.setVerticalDegree((short) 10); //Vertical up and down movement angle 10

relativeAngleDesktopMotion.setVerticalSpeed((short) 10); // Speed control 1-60 upward movement, positive number upward movement, negative number downward movement, from positive to

negative speed controls the up and down movement direction

relativeAngleDesktopMotion.setRollDegree((short) 10); //Rolling angle 10

relativeAngleDesktopMotion.setRollSpeed((short) 10); // Speed control 1-60 Roll to the right, positive number means right, negative number means left,

Speed controls the rolling and left and right movement directions

desktopMotionManager.doRelativeAngleMotion(relativeAngleDesktopMotion);
```

3.15.4 Absolute Angle Movement of Desktop

Method signature:

public OperationResult

doAbsoluteAngleMotion(AbsoluteAngleDesktopMotion absoluteAngleDesktopMotion)

Method Description:

Send instructions to the background service directly through **the desktopManager** object. After receiving the instruction, the background service will

Perform the corresponding operation.

Absolute angle control, speed can only be positive

Vertical angle control range: 0-45

Horizontal angle control range: 0-160

Roll angle control range: 0-160

Front and rear angle control range: 0-45

Sample code:

```
AbsoluteAngleDesktopMotion absoluteAngleDesktopMotion=new AbsoluteAngleDesktopMotion();
    absoluteAngleDesktopMotion.setForwardDegree((short) 10); //The required forward and backward movement angle is 10
    absoluteAngleDesktopMotion.setForwardSpeed((short) 10);
    absoluteAngleDesktopMotion.setHorizontalDegree((short) 10); //Horizontal movement angle 10
    absoluteAngleDesktopMotion.setHorizontalTurnSpeed((short) 10);
    absoluteAngleDesktopMotion.setVerticalDegree((short) 10); //Vertical up and down movement angle 10
    absoluteAngleDesktopMotion.setVerticalSpeed((short) 10);
    absoluteAngleDesktopMotion.setRollDegree((short) 10); //Rolling angle 10
    absoluteAngleDesktopMotion.setRollSpeed((short) 10);
desktopMotionManager.doAbsoluteAngleMotion(absoluteAngleDesktopMotion);
```